CHAPTER 29

Errors, Retries, Timeouts and Backoff

In this chapter

Introduction: Five Generations for One Answer . . . 3
29.1 The Failure Taxonomy . . . 4
29.2 What Is Not an Error . . . 6
29.3 Timeouts: Yours, Theirs, and the Gap Between . . . 6
29.4 The Uncertainty at the Heart of a Retry . . . 7
29.5 Backoff, and Why Jitter Matters More Than the Exponent . . . 8
29.6 The Retry Budget . . . 8
29.7 What a Retry Costs . . . 9
29.8 Visual Analogies . . . 9
29.9 Architecture: One Call, All Its Branches . . . 10
29.10 Examples . . . 10
29.11 Code Walkthrough: The Gate That Refuses to Guess . . . 11
29.12 Mini Project: The Retry Bench . . . 13
29.13 Panel A — The Catalogue, and What a Naive Policy Spends on Nothing . . . 13
29.14 Panel B — A Timeout Does Not Cancel the Generation . . . 14
29.15 Panel C — Five Thousand Clients and One Outage . . . 15
29.16 Panel D — A Budget Is Fleet-Wide; max_retries Is Not . . . 16
29.17 Evaluation and Deployment . . . 17
29.18 Hands-on Labs . . . 17
29.19 Lab Solutions . . . 17
29.20 Best Practices . . . 18
29.21 Common Mistakes . . . 20
29.22 Performance Considerations . . . 22
29.23 Security Considerations . . . 22
29.24 Real Production Examples . . . 23
29.25 Certification Notes . . . 23
29.26 Twenty Practice Questions . . . 24
29.27 Answers and Explanations . . . 25
29.28 Ten Practical Exercises . . . 26

29.29 Five Challenge Labs 26
29.30 Ten Interview Questions 26
29.31 Summary 27
29.32 Glossary 28
29.33 Further Reading 29

Part 5 of 13 · Chapter 29 of 64. Next: Chapter 30 — Batch Processing, Prompt Caching and Cost Controls.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Classify every documented HTTP failure as retryable or not, and refuse to guess about anything unlisted.
2. Explain why a stop reason is never an error, why a refusal must not be retried on the same model, and why an error can still arrive inside an HTTP 200.
3. Quantify what a client-side timeout costs when it does not cancel the work, and choose a timeout from your own latency distribution rather than from an example.
4. Say what the Claude API does and does not promise about duplicate requests, and design for the absence of an idempotency key.
5. Implement exponential backoff with jitter, and say why the jitter matters more than the exponent.
6. Honour `retry-after` and explain the documented consequence of not doing so.
7. Impose a fleet-wide **retry budget**, and explain why `max_retries` cannot do that job.
8. Distinguish the four things a retry can cost: latency, rate limit, money, and a duplicated effect.

NOTE

Prerequisites

Chapter 16 for the request and `stop_reason`. **Chapter 17** for the `error` event that arrives inside an HTTP 200. **Chapter 19** for rate limits, the token bucket, `retry-after` and the retry bake-off measured there. **Chapter 25** for refusals as successful responses. **Chapter 26** for the error catalogue, the SDK defaults `max_retries=2` and `timeout=600`, and the RPM amplification they cause. **Chapter 27** for `max_tokens` and the 400s a body can earn. **Chapter 28** for the response reader that decides whether there is anything to retry.

Deferred by name: **Batch processing**, which removes most of this problem by replacing a connection with a poll, is **Chapter 30**. **Circuit breakers and bulkheads** as production patterns are **Chapter 56**. **Observability and tracing** are **Chapter 57**. **Orchestrator-level replanning** after a failed step is **Chapter 51**.

Introduction: Five Generations for One Answer

The Anthropic Python SDK documentation contains this example, offered to show how the timeout option works:

```
client = Anthropic(
    timeout=20.0,   # 20 seconds (default is 10 minutes)
)
```

It is a correct example of the API. Section 29.15 measures what happens when it meets a real workload.

Take a service whose server-side generation time is log-normal with a median of **45 seconds** — unremarkable for a long summarisation or a thinking-enabled analysis; its p90 is 144 seconds and its p99 is 371. Set `timeout=20.0` and leave `max_retries` at its default of 2. Then:

- **81.61 per cent** of attempts exceed the timeout,
- the client makes **2.482** attempts per logical request,
- **54.36 per cent** of logical requests end with no answer at all,
- and for every answer that *is* delivered, **5.439 generations run on the server**.

Five and a half generations per answer, and more than half of requests failing, from one line that looks like configuration hygiene. Raise the timeout to the SDK's default of 600 seconds and the same workload runs **1.002** generations per answer with a **0.002 per cent** failure rate.

The reason it is so bad is the sentence nobody writes down: **a client-side timeout does not cancel the generation.** Closing the connection is a decision your process makes about waiting. The server has no idea and keeps going. Every abandoned attempt is work that happened, that finished, and that nobody will ever read.

And you cannot deduplicate it, because there is nothing to deduplicate with. The Claude API documents no idempotency key on `POST /v1/messages`. It is not documented as absent either — which by this book's rule from Chapter 14 makes it **UNKNOWN**, and the conservative reading is the only safe one: assume a retried request is a second request, because nothing in the protocol says otherwise. The documentation does supply a precedent for the shape of the problem, on a different endpoint: the Claude Code routine-fire endpoint states plainly that it has no idempotency key and that a retrying webhook caller creates multiple sessions.

That is the first of this chapter's four measurements. The second is about which failures are worth retrying at all. Section 29.14 takes a realistic mix of ten thousand errors during an incident — a bad deploy producing 400s, one expired key, some oversized documents, and a great many 429s, 500s and 529s — and runs it past two policies. A policy that retries anything that threw sends **20,000** retry requests, of which **6,600 — 33.0 per cent — could not possibly succeed**, because a 400 is still a 400 the second time. Every one of those draws on the RPM bucket Chapter 19 measured, during the exact minute you can least afford it.

The third is about timing rather than counting. Section 29.16 puts five thousand clients through a thirty-second outage. All of them fail at the same instant, so all of them back off from the same instant, and without jitter they retry in lockstep. When the service recovers at 500 requests per second it is hit by **5,000 requests in a single second**, serves 500, and the other 4,500 back off together and do it again. **40.0 per cent of the fleet is ever served**, and the synchronised fleet sends **42,000** requests to achieve that. Add equal jitter and the same fleet sends **34,998** requests, peaks at **339** per second — **14.7 times lower** — and **100 per cent** of clients are served.

The fourth is the one no SDK can do for you. `max_retries` is a property of a client, and an incident is a property of a fleet. At a 30 per cent failure rate, per-client retries multiply your load by **1.6027**; a fleet-wide budget capping retries at ten per cent of real traffic holds it at **1.1000**. Under a total outage the difference is **3.00× against 1.10×**.

Chapter 28 covered what to do when the call succeeds. This chapter covers the other branch, and its through-line is a single question that the word "retry" tends to suppress: **what did the first attempt already do?**

29.1 The Failure Taxonomy

Chapter 26 introduced the status codes as part of the front door. Here they are again as a control-flow table, with the one property that decides everything downstream.

Status	Type	Retryable	Why
400	`invalid_request_error`	**no**	the request itself is wrong
401	`authentication_error`	**no**	the key is malformed, revoked or expired
402	`billing_error`	**no**	billing or payment information
403	`permission_error`	**no**	the key lacks permission
404	`not_found_error`	**no**	the path or a resource id is wrong
408	`request_timeout`	yes	retried by the SDKs by default
409	`conflict_error`	yes	concurrent modification; resolve, then retry
413	`request_too_large`	**no**	over the ceiling; it will be next time too
429	`rate_limit_error`	yes	honour `retry-after`
500	`api_error`	yes	retry with exponential backoff
504	`timeout_error`	yes	timed out while processing
529	`overloaded_error`	yes	temporarily overloaded across all users
—	connection error	yes	**no response was received**

Six retryable, six not, and one that is not an HTTP status at all.

> DEFINITION
>
> **Definition — retryable**
>
> A failure is **retryable** when the cause of the failure is something that changes on its own. A 429 is retryable because a minute passes. A 400 is not retryable because nothing about the passage of time makes a malformed body well-formed. "Retryable" is a statement about the *cause*, not about your willingness to try again.

Two of the rows deserve a note.

409 `conflict_error` is retryable in a different sense from the rest: the documented advice is *"resolve the conflict, then retry the request"*, which means re-reading state rather than sleeping. A retry loop that treats it like a 429 will conflict forever.

A connection error has no status code, and it is the most dangerous entry in the table, because it is the one case where **you do not know whether the request was processed**. That uncertainty is the subject of §29.4.

And the catalogue is open. Chapter 26 recorded the versioning contract: *"it is possible that the* `type` *values will grow over time."* A classifier that returns `False` for an unrecognised status is guessing; §29.13's raises

instead.

29.2 What Is Not an Error

Three things look like failures and are not, and each needs a different response.

A stop reason is a successful response. Chapter 28's whole table lives in the body of an HTTP 200. `max_tokens` is not an error, and retrying it unchanged produces the same truncation.

A refusal is a successful response, and retrying the same model is the one thing that cannot help. Chapter 25 established the contract: HTTP 200, `stop_reason: "refusal"`, possibly empty `content`. The documented remedy is a **different model**, server-side or client-side, with fallback credit to avoid paying for the cached prefix twice. A retry loop that catches exceptions will never see a refusal at all — Chapter 25 measured an error-rate monitor seeing **0 of 6**.

An error can arrive inside an HTTP 200. Chapter 17 recorded that an `overloaded_error` can appear as an `error` event mid-stream. The status line said 200, the tokens already streamed have been produced, and a retry is a fresh generation rather than a resumption. Chapter 17's recovery rules apply: on 4.5 and earlier resend the partial as an assistant message; on 4.6 and later send it inside a **user** message with an instruction to continue, because prefilling is gone.

> WARNING
>
> **The three-way branch that most code gets wrong**
>
> `try/except` around the call catches only the third of the three cases below.
>
> 1. **HTTP 4xx/5xx** — an error. Classify it.
> 2. **HTTP 200 with a stop reason** — a successful response. Read it (Chapter 28).
> 3. **HTTP 200 with an `error` event inside the stream** — neither. Handle it explicitly.
>
> A refusal is case 2 and needs a fallback model. A 429 is case 1 and needs a delay. Nothing sensible happens if you route them both through the same handler.

29.3 Timeouts: Yours, Theirs, and the Gap Between

There are three timeouts in play and they belong to different parties.

Timeout	Owner	Default	What happens on expiry
client-side	your HTTP client	**10 minutes** in the SDKs	the connection is closed; **the server keeps going**
server-side	Anthropic	—	**504** `timeout_error`
idle connection	the network	varies	the connection drops, often silently

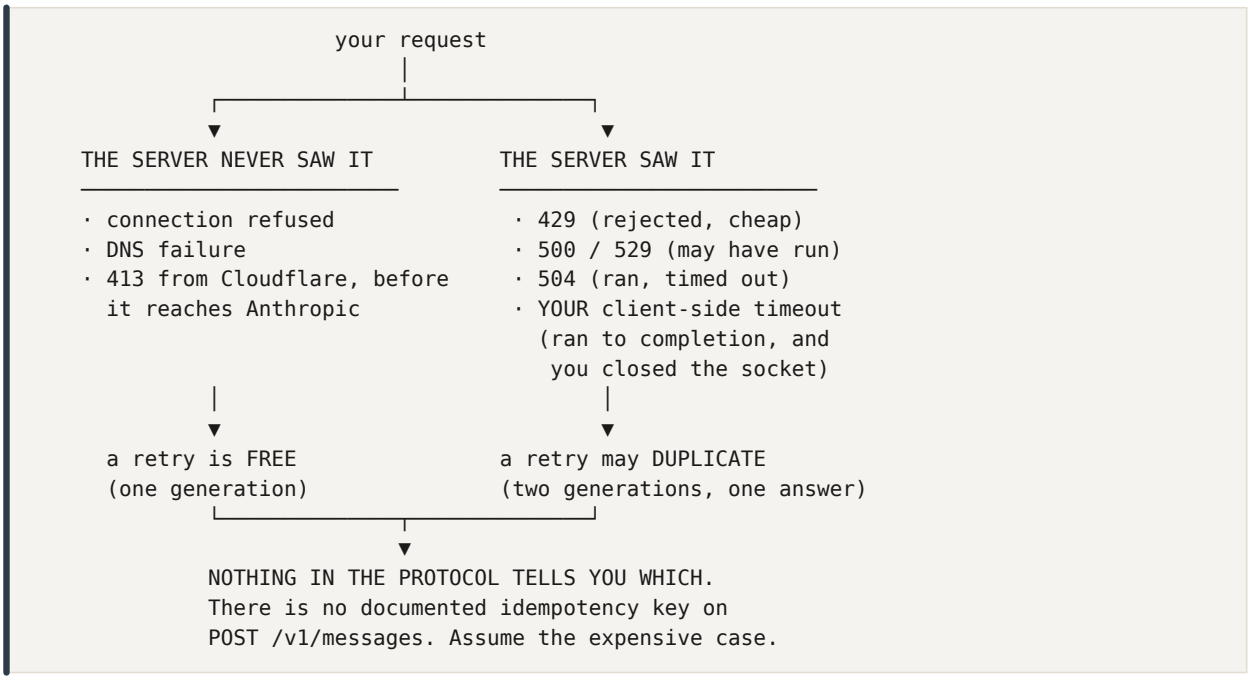
The SDKs add two guards worth knowing. They *"validate that your non-streaming Messages API requests are not expected to exceed a 10-minute timeout"*, throwing a `ValueError` before the request leaves — a client-side check, not an API limit, disabled by passing `stream=True` or overriding `timeout`. And they set a

TCP socket keep-alive specifically to reduce idle-connection drops, which is one reason to prefer `DefaultHttpxClient` over a raw client if you replace it.

The documented advice for long work is unambiguous and appears on three separate pages: **stream it, or batch it.** *"Some networks may drop idle connections after a variable period of time, which can cause the request to fail or time out without receiving a response."* Chapter 30 takes the Batch route, which replaces a held connection with a poll and removes this entire class of failure.

29.4 The Uncertainty at the Heart of a Retry

When a call fails, there are exactly two situations, and most failures do not tell you which one you are in.



The bottom-right box is the one this chapter is about. A client-side timeout is the purest form of it: the request was valid, the server accepted it, generation ran to completion, and the only thing that went wrong is that your process stopped waiting. There is no cancellation, so the work is done. $29.15 prices it.

Is there an idempotency key? The Messages API reference documents none, and the guides do not mention one. Chapter 23's rule applies unchanged — **a fact documented in neither the supported nor the unsupported list is UNKNOWN** — and here the conservative reading is also the operationally correct one: design as though a retry is a second request, because nothing promises otherwise. The documentation does describe this hazard explicitly for a different endpoint, noting that the Claude Code routine-fire endpoint has no idempotency key and that a retrying caller creates multiple sessions. The shape of the problem is acknowledged; the guarantee is simply not offered.

Two consequences for design.

Make the effect idempotent, since the request is not. If a completion writes a row, key the row on something you generate before the call and can reuse on the retry. Chapter 34 makes the same argument about tool results.

Budget for duplicate work, not just duplicate latency. Chapter 26 counted retries against the RPM bucket. This chapter counts them against the invoice, and the multiplier is the same arithmetic pointed at a different meter.

29.5 Backoff, and Why Jitter Matters More Than the Exponent

Exponential backoff is the standard construction: wait `base × factor^(n−1)` before attempt *n*, capped. The SDKs use *"a short exponential backoff"* with **two retries by default**, honouring `retry-after`.

The exponent is the famous part and the least important one. What matters is that a fleet fails **together** — an outage is common-mode by definition — so every client starts its backoff clock at the same instant and, without randomisation, they all wake up at the same instant too.

Mode	Delay before attempt n
none	`min(cap, base·factor^(n−1))`
full jitter	`uniform(0, min(cap, base·factor^(n−1)))`
equal jitter	`half + uniform(0, half)`, where `half` is half the raw delay

Section 29.16 measures the difference at **14.7×** on peak arrival rate, and — the part that surprises people — the synchronised fleet **sends more requests and gets fewer of them served**.

> TIP
>
> **`retry-after` is not advice**
>
> Chapter 19 recorded the documented consequence in the API's own words: **"earlier retries will fail."** When the header is present it replaces your backoff calculation entirely. Sleeping less than it says converts a delay into a second 429, and Chapter 19's bake-off measured what a fleet doing that costs.

29.6 The Retry Budget

`max_retries` is a property of one client object. An incident is a property of a fleet, and the two do not compose: a hundred processes each politely retrying twice is a fleet retrying two hundred times.

The standard fix is a **budget**: allow a retry only while the running count of retries is below some share — ten per cent is a common choice — of the running count of real requests. The effect is that retry load is bounded by your *success* volume rather than by your *failure* volume, so a total outage cannot amplify traffic at all.

Failure rate	Unbudgeted load multiplier	Budgeted at 10%
5%	1.1006×	1.0997×
30%	**1.6027×**	**1.1000×**
100%	3.0000×	1.1000×

At a five per cent failure rate the budget is invisible, which is the property you want: it costs nothing in normal operation and only binds when things are already bad. Chapter 26 showed the same 30 per cent episode pushing a plan from 96.00 per cent of its RPM bucket to 133.44 per cent; a budget is what stops that.

29.7 What a Retry Costs

Four meters, and most retry code is written as though there were one.

Meter	What a retry adds
Latency	the backoff, plus the failed attempt. Chapter 26: 2 retries × 600 s = a **30-minute** worst case
Rate limit	a full request against RPM, and its input tokens against ITPM (Chapter 19)
Money	a second generation if the first one completed — $29.15
Effect	a duplicated side effect, if the completion did anything

The fourth is the one with no dial. Latency you can bound, rate limit you can budget, money you can cap. A duplicated effect is a correctness problem, and the only place to fix it is in your own code, because the API offers no idempotency key to fix it in the protocol.

29.8 Visual Analogies

VISUAL ANALOGY

Hanging up on a chef

A client-side timeout is putting the phone down on a restaurant mid-order. You have stopped waiting. The kitchen has not stopped cooking. Ring back and order again and two meals are made, one of which nobody eats.

Where it breaks. A restaurant would recognise your voice and ask. The API cannot: there is no idempotency key, so the second call is a genuinely new order as far as anything in the protocol can tell. And unlike a kitchen, the cost of the abandoned meal scales with how long it took — the slowest requests are exactly the ones most likely to be abandoned, and the most expensive.

VISUAL ANALOGY

A room of people all told to wait five minutes

Exponential backoff without jitter is a queue that is turned away and told to come back in five minutes. Everyone comes back in exactly five minutes. The door reopens, a few get in, the rest are told five more, and they come back together again.

Where it breaks. Real people vary — someone stops for coffee. Machines do not vary at all unless you make them, which is the whole content of the word jitter. The analogy also understates the cost: the synchronised crowd does not merely wait longer, it **sends more requests and gets fewer served** — 42,000 requests for a 40 per cent success rate, against 34,998 for 100 per cent.

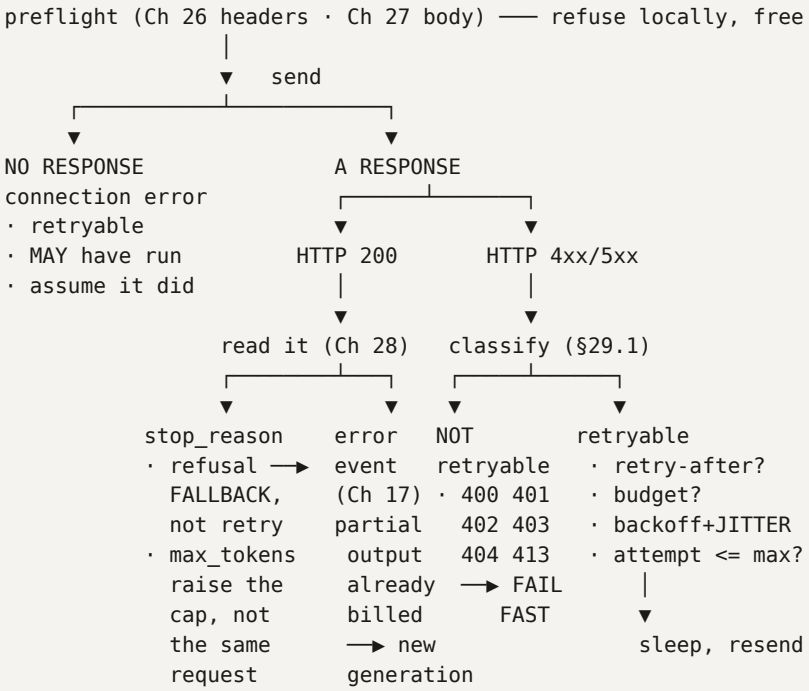
VISUAL ANALOGY

A fire brigade with a water budget

`max_retries` is each firefighter deciding to make two extra trips. A retry budget is the incident commander capping total draw at ten per cent of the mains. When one building is alight the cap never binds. When the whole street is alight it is the only thing keeping pressure in the pipes.

Where it breaks. Water is conserved and requests are not — an unretried request does not go anywhere useful. So a budget genuinely does drop work on the floor, and the argument for it is not that nothing is lost but that a **capped, known** loss beats an uncapped one that takes the service down for everybody. That is a trade to make deliberately, in advance.

29.9 Architecture: One Call, All Its Branches



Three rules the picture encodes. **Preflight first**, because a locally refused request costs nothing and a 400 costs a round trip and a rate-limit slot. **The 200 branch is not the error branch** — a refusal needs a different model and a truncation needs a different `max_tokens`, and neither is helped by sending the identical bytes again. And **every retryable path passes through the same four gates**: `retry-after`, budget, jitter, attempt count. Putting them anywhere but one place is how a fleet ends up with four different retry policies.

29.10 Examples

Example 1 — the classifier, which is most of the value.

```
RETRYABLE = {408, 409, 429, 500, 504, 529}
NOT_RETRYABLE = {400, 401, 402, 403, 404, 413}
def retryable(status):
```

```
    if status is None:          # connection error: no response
        return True
    if status not in RETRYABLE | NOT_RETRYABLE:
        raise Unverified(f"HTTP {status} is not in the catalogue")
    return status in RETRYABLE
```

Example 2 — backoff with jitter, three lines.

```
def delay(attempt, base=1.0, factor=2.0, cap=60.0):
    raw = min(cap, base * factor ** (attempt - 1))
    return random.uniform(0.0, raw)          # full jitter
```

Example 3 — `retry-after` replacing the calculation entirely.

```
wait = response.headers.get("retry-after")
sleep = float(wait) if wait is not None else delay(attempt)
```

Example 4 — an idempotent effect, since the request is not. Generate the key **before** the call so the retry reuses it:

```
effect_key = uuid.uuid4().hex          # once, outside the retry loop
for attempt in range(1, max_attempts + 1):
    msg = call()
    if msg is not None:
        db.upsert(effect_key, msg)     # a duplicate call is now safe
        break
```

Example 5 — the three-way branch, in the right order.

```
try:
    resp = call()
except APIStatusError as e:
    handle_error(e.status_code)        # §29.1
else:
    reading = read(resp)               # Chapter 28
    if reading.action == "FALLBACK":
        retry_on_another_model(resp)   # Chapter 25, NOT a retry
    elif reading.action == "TRUNCATED":
        raise_the_cap_and_resend()     # Chapter 27, NOT a retry
```

Example 6 — turning the problem off. For anything long, the documented answer is not a better timeout:

```
with client.messages.stream(model="claude-opus-5", max_tokens=128_000,
                            messages=msgs) as stream:
    message = stream.get_final_message()   # no idle connection risk
```

29.11 Code Walkthrough: The Gate That Refuses to Guess

`errors.py` is a table and one function, and the function is three lines longer than it needs to be for a reason.

```
def classify(status):
    if status is None:
        return CONNECTION_ERROR[1]
    if status not in TYPES:
        raise Unverified(
            f"HTTP {status} is not in the catalogue verified on "
            f"{VERIFIED_ON}; the documented `type` values may grow, "
            f"so this must be handled explicitly")
    return status in RETRYABLE
```

Line 2 handles the case with no status at all, and returns `True` — a connection error is retryable, and §29.4 explains why it is also the most dangerous kind. Line 4 raises rather than returning `False`. The temptation is enormous: `return status in RETRYABLE` alone is one line, never raises, and treats every unfamiliar status as non-retryable, which sounds conservative. It is not conservative, it is silent. Chapter 26 recorded that the documented `type` values *"may grow over time"*, and the day a new retryable status appears, the one-line version quietly stops retrying it and nobody finds out.

```
class Budget:
    def allow(self):
        if self.retries + 1 > self.share * max(self.requests, 1):
            return False
        self.retries += 1
        return True
```

The whole budget is one comparison. `self.requests` counts real requests and `self.retries` counts retries, and the retry is permitted only while the ratio is under the share. The consequence is worth stating precisely: under a **total** outage the number of real requests keeps rising while none succeed, so the budget converges to exactly `share` — §29.18 measures 1.1000× against an unbudgeted 3.0000×.

```
def backoff_delays(attempt, base=1.0, factor=2.0, cap=60.0,
                   jitter="full", rng=None):
    raw = min(cap, base * (factor ** (attempt - 1)))
    if jitter == "none":
        return raw
    ...
    if jitter == "full":
        return rng.uniform(0.0, raw)
```

Line 3 is the exponent everybody talks about. Line 8 is the part that matters. Note also that `jitter="none"` is available and tested — this book measures the bad option rather than omitting it, and §29.16 is the result.

```
        if attempt > self.max_retries:
            return None
        if not classify(status):
            return None
        if self.budget is not None and not self.budget.allow():
            return None
        if retry_after is not None and self.honour_retry_after:
            return float(retry_after)
```

Four gates, in this order, and the order is deliberate. The attempt count is checked first because it is free. The classifier is next because a 400 must never reach the budget and consume it. The budget is next

because a spent budget should stop a retry even when the server has told us when to come back. And `retry-after` is last because when it is present it **replaces** the backoff calculation entirely rather than being combined with it.

29.12 Mini Project: The Retry Bench

What it is. A classifier, a policy and three simulations that between them answer the question a retry wrapper never asks: what did the first attempt already do?

```
ch29/
  reader.py     copied from ch28/ — decides if there is anything to retry
  fixtures.py   copied from ch28/ — the response corpus
  errors.py     the documented catalogue; classify() raises on the unlisted
  retry.py      backoff, jitter, retry-after, and a fleet-wide Budget
  simulate.py   duplicate work · the herd · the budget
  report.py     prints every figure in this chapter
  test_ch29.py  28 tests
```

```
   status ──► errors.classify ──┬─► True / False
                                └─► Unverified (raises)
   error mix ──► errors.wasted_retries ──► retries that cannot succeed
   latency distribution + timeout ──► simulate.duplicate_work
                                        ├─ p(attempt exceeds)
                                        ├─ attempts per request
                                        └─ GENERATIONS PER ANSWER
   5,000 clients + outage ──► simulate.herd(jitter=...)
                                        ├─ peak/s after recovery
                                        └─ served, p95
   failure rate ──► simulate.budgeted_load ──► load multiplier
```

```
cd ch29 && python3 report.py        # every figure below
cd ch29 && python3 -m pytest -q     # 28 passed
```

Deployment. One policy object, constructed once, shared by every call site — because four call sites with four policies is four incidents. The `Budget` belongs in a process-wide singleton at minimum and in a shared counter if your fleet can afford one; a per-process budget already bounds each process, which is most of the benefit.

29.13 Panel A — The Catalogue, and What a Naive Policy Spends on Nothing

Thirteen documented outcomes: **six retryable, six not**, and one with no status code at all.

A realistic incident mix, BUILT to exercise the classifier — a bad deploy sending malformed bodies, one expired key in the fleet, some oversized documents, and the 429s, 500s and 529s that come with the territory:

Failure	Count	Retryable?
400 `invalid_request_error`	3,000	**no**

Failure	Count	Retryable?
401 `authentication_error`	100	**no**
413 `request_too_large`	200	**no**
429 `rate_limit_error`	4,000	yes
500 `api_error`	1,200	yes
529 `overloaded_error`	1,000	yes
connection error	500	yes

	Requests
errors	10,000
retry requests sent by "retry anything that threw"	**20,000**
retries that could possibly succeed	13,400
wasted	**6,600 — 33.0%**

A third of the retry traffic is spent re-sending requests that were wrong the first time and are still wrong. Two things make that worse than the raw number looks. Those 6,600 requests arrive during an incident, when the RPM bucket is already the binding constraint — Chapter 19's headline was that retries and real traffic draw from the same bucket. And they arrive *fast*, because a 400 comes back immediately: the non-retryable failures are the cheapest to produce and therefore the quickest to re-send.

KEY TAKEAWAYS

The finding in one line

A third of a naive policy's retries are arithmetically incapable of succeeding, and they arrive at the worst possible moment, at the highest possible rate.

29.14 Panel B — A Timeout Does Not Cancel the Generation

Server-side generation latency modelled as log-normal, median **45 s**, p90 **144 s**, p99 **371 s**. Client `max_retries` at its default of 2, so up to three attempts.

Timeout	p(attempt exceeds)	Attempts per request	Generations per answer delivered	No answer at all
20 s *(the docs' example)*	**0.8161**	2.482	**5.439**	**54.36%**
60 s	0.3751	1.516	1.600	5.28%
120 s	0.1388	1.158	1.161	0.27%
300 s	0.0176	1.018	1.018	0.00%

Timeout	p(attempt exceeds)	Attempts per request	Generations per answer delivered	No answer at all
600 s *(the SDK default)*	**0.0020**	1.002	**1.002**	**0.00%**

Read the fourth column, because it is the one that does not appear on any dashboard. At a 20-second timeout the service produces **5.439 completed generations for every answer it manages to deliver**. Four and a half of those are work the server did, finished, and threw away, because the client had already closed the socket and gone back to the queue.

Two mechanisms compound to produce that number. Only 45.64 per cent of logical requests deliver an answer at all, so the denominator is small. And every abandoned attempt runs to completion regardless, so the numerator counts all 2.482 attempts. Dividing one by the other is what turns a bad timeout into a five-fold multiplier.

The shape of the curve matters more than any single row. Between 20 and 120 seconds the cost falls by a factor of nearly five; between 300 and 600 it barely moves. **The timeout is a cliff, not a dial**, and the cliff sits at roughly the p90 of your own latency distribution — which is why the only defensible way to choose it is to measure that distribution rather than to copy a number out of an example.

COMMON MISTAKE

Copying `timeout=20.0` out of the documentation

Symptom. A high failure rate with no errors in the API's own metrics, and a bill that does not match the number of answers produced. **Cause.** A client-side timeout below the workload's p90. The server completes every abandoned generation. **Fix.** Set the timeout from your measured latency distribution, or stream, or batch. **Detection.** Compare completed generations against delivered answers. If you cannot, compare spend against answers — at a 20-second timeout the ratio here is **5.439**.

29.15 Panel C — Five Thousand Clients and One Outage

Five thousand clients, all failing at t = 0. The service recovers at **t = 30 s** with a capacity of **500 requests per second**. Backoff base 1 s, factor 2, cap 60 s, up to 8 retries, 300-second horizon.

Jitter	Requests sent	Peak/s after recovery	Served	p95 served at
none	**42,000**	**5,000**	**2,000 (40.0%)**	**never**
equal	34,998	**339**	**5,000 (100.0%)**	55.7 s
full	37,595	216	4,980 (99.6%)	77.6 s

The synchronised fleet **sends 20 per cent more requests and gets 60 per cent fewer clients served.** That is the finding, and it is worth being precise about the mechanism rather than waving at "thundering herd".

Without jitter, every client computes the same delay from the same failure instant, so the arrival pattern is a sequence of impulses: 5,000 requests in the second at t = 31, of which 500 succeed; 4,500 in the second at t = 63; and so on. Capacity is 500 per second and the demand arrives in one second, so **90 per**

cent of each burst is refused no matter how much total capacity exists over the window. The service is not overloaded on average — it has 300 seconds × 500/s of capacity for 5,000 clients — it is overloaded for one second at a time, eight times.

With jitter the same 5,000 clients spread across the window, the peak falls to 339 — a factor of **14.7** — and every client is served.

The comparison between the two jitter modes is worth publishing rather than smoothing over. **Equal jitter beats full jitter here** on both peak and p95, because full jitter's `uniform(0, raw)` pulls many retries *earlier*, into the outage window where they are wasted; equal jitter's floor of half the raw delay keeps them out of it. Full jitter is the better-known recommendation and is not universally better; which wins depends on the ratio of outage length to backoff schedule, and the honest answer is that you should simulate your own.

KEY TAKEAWAYS

The finding in one line

Jitter is not a refinement of backoff; it is the part that works. Without it a fleet with ample aggregate capacity serves 40 per cent of its clients while sending more requests than the fleet that serves all of them.

29.16 Panel D — A Budget Is Fleet-Wide; `max_retries` Is Not

Failure rate	Unbudgeted (`max_retries=2`)	Budgeted at 10%
5%	1.1006×	1.0997×
30%	**1.6027×**	**1.1000×**
100%	**3.0000×**	**1.1000×**

Three readings.

At a low failure rate the budget is invisible. 1.1006 against 1.0997 — a difference of one part in a thousand. A control that costs nothing when things are fine and binds only when they are not is the rare kind you can leave switched on.

At 30 per cent it is the difference between coping and not. Chapter 26 measured the same episode taking a plan from 96.00 per cent of its RPM bucket to **133.44 per cent**; the budget holds it at 105.6 per cent of the original 96, which is still inside the bucket.

Under a total outage it is the only thing that bounds you at all. Unbudgeted, every request becomes three: a 3.0× load multiplier delivered precisely when the service is least able to absorb it, which is the classic mechanism by which an outage extends itself past its own cause. Budgeted, the multiplier is 1.1 regardless.

The reason `max_retries` cannot do this job is structural rather than a deficiency of the SDKs. `max_retries` bounds the retries of **one request**. A budget bounds the retries of **all requests**, as a share of throughput. Those are different quantities, and only the second is what an overloaded service experiences.

29.17 Evaluation and Deployment

Measure	Value
Documented statuses catalogued	13 — 6 retryable, 6 not, 1 with no status
Unlisted statuses	raise, rather than defaulting to non-retryable
Timeout sweep	5 settings, worst **5.439** generations per answer
Jitter comparison	3 modes, **14.7×** peak reduction
Budget comparison	3 failure rates, **3.00×** → **1.10×** at total failure
Tests	28, passing, no key, no network

Deployment has an order, and it is the reverse of how these things usually get built. **The classifier first**, because it removes a third of the retry traffic for one afternoon's work. **Then the timeout**, measured from your own distribution rather than copied. **Then jitter**, which is three lines and the largest single effect in this chapter. **Then the budget**, which is the only one that needs coordination and the only one that helps when everything is already on fire.

29.18 Hands-on Labs

Lab 29.1 — Build the classifier. Write `retryable(status)` over the thirteen documented outcomes. *Success criterion:* an unlisted status raises, and your test asserts the exception type rather than a boolean.

Lab 29.2 — Price your own timeout. Take your service's real latency distribution and compute generations-per-answer at four candidate timeouts. *Success criterion:* you can name the timeout below which your cost per answer more than doubles.

Lab 29.3 — Add jitter and measure it. Simulate a fleet through an outage with and without jitter. *Success criterion:* your peak arrival rate falls by more than 5×, and you can explain why the synchronised run sends *more* requests.

Lab 29.4 — Impose a budget. Add a retry budget to one service and drive it to a 100 per cent failure rate in a test. *Success criterion:* the load multiplier stays within a few per cent of `1 + share`.

Lab 29.5 — Make an effect idempotent. Take one call whose completion writes something, and key the write on an identifier generated before the call. *Success criterion:* running the call twice with the same key produces one row.

29.19 Lab Solutions

29.1. Two sets and a membership test, plus a raise for anything unlisted. The raise is the whole exercise: returning `False` for an unknown status looks conservative and is silent, and Chapter 26's versioning contract says the list will grow. See `errors.classify`.

29.2. Generations per answer is `attempts ÷ (1 − p^(retries+1))` where `p = P(latency > timeout)` and `attempts = Σ p^k`. The two terms move in opposite directions as the timeout falls — more attempts, fewer answers — which is why the curve is so steep. At the numbers in Panel B the ratio goes from 1.002 at 600 s to **5.439** at 20 s. See `simulate.duplicate_work`.

29.3. The synchronised fleet sends more because it *fails* more: each impulse exceeds capacity by 10×, so 90 per cent of each burst retries again. Jitter does not add capacity; it stops the fleet from wasting the capacity that exists. See `simulate.herd`.

29.4. One comparison — `retries + 1 > share × requests` — and a counter of real requests that keeps rising whether or not they succeed. That is what makes the multiplier converge to `1 + share` under total failure rather than to `1 + max_retries`. See `retry.Budget`.

29.5. Generate the key **outside** the retry loop. The commonest bug is generating it inside, which makes every attempt idempotent with respect to itself and with respect to nothing else.

29.20 Best Practices

BEST PRACTICE

Classify before you retry, and raise on the unlisted

Six statuses are retryable and six are not. A policy that retries anything that threw wastes **33.0 per cent** of its retries on requests that cannot succeed.

BEST PRACTICE

Choose the timeout from your own latency distribution

Not from an example. The cost curve is a cliff near your p90, and below it the generations-per-answer ratio rises fast — **5.439** at 20 seconds on the workload in Panel B.

BEST PRACTICE

Always jitter

It is three lines and the largest single effect in this chapter: a **14.7×** lower peak and 100 per cent of clients served instead of 40 per cent. Simulate to choose between full and equal jitter; the better-known recommendation is not always the better one.

BEST PRACTICE

Honour `retry-after` exactly

The documentation states that **earlier retries will fail**. When the header is present it replaces your backoff, it does not combine with it.

BEST PRACTICE

Impose a fleet-wide retry budget

`max_retries` bounds one request; a budget bounds all of them. It is invisible at a 5 per cent failure rate and the difference between 3.0× and 1.1× at total failure.

BEST PRACTICE

Assume a retried request is a second request

No idempotency key is documented on `POST /v1/messages`. Make the **effect** idempotent, with a key generated before the first attempt.

BEST PRACTICE

Never retry a refusal on the same model

It is an HTTP 200, not an error, and the documented remedy is a different model — with fallback credit so the cached prefix is not paid for twice (Chapter 25).

BEST PRACTICE

Never resend an identical request after `max_tokens`

It is not an error either. Raise the cap, or continue the response. Chapter 27 sized the cap; Chapter 28 labelled the truncation.

BEST PRACTICE

Stream or batch anything long, instead of tuning the timeout

The documented advice on three separate pages. Batch replaces a held connection with a poll and removes this whole class of failure — Chapter 30.

BEST PRACTICE

Put the policy in one place

Middleware (Chapter 26), constructed once. Four call sites with four retry policies is four incidents waiting for their turn.

BEST PRACTICE

Log the attempt number and the request ID of every attempt

`request-id` is on every response (Chapter 26). Without the attempt number you cannot tell a slow request from a third try, which is the single most useful distinction during an incident.

BEST PRACTICE

Alert on p100 latency exceeding your configured timeout

If it does, you are looking at retries. It is the cheapest possible detector and it costs one alert rule.

29.21 Common Mistakes

COMMON MISTAKE

Retrying anything that threw

Symptom. RPM exhausted during an incident caused by something else entirely. **Cause.** No classifier. A 400 is retried three times and fails three times. **Fix.** Six retryable statuses, six not, and a raise for the unlisted. **Detection.** Panel A: **6,600 of 20,000** retry requests could not succeed.

COMMON MISTAKE

Copying a timeout out of an example

Symptom. A high failure rate that the API's own error metrics do not show. **Cause.** A client-side timeout below the workload's p90; the abandoned generations complete anyway. **Fix.** Measure the distribution. Or stream. Or batch. **Detection.** Panel B: **5.439** generations per answer at 20 seconds.

COMMON MISTAKE

Assuming a timeout cancels the request

Symptom. Spend that does not track delivered answers. **Cause.** Closing a socket is a decision your process makes; the server is not told. **Fix.** Treat every timed-out attempt as completed work, and design the effect to be idempotent. **Detection.** Reconcile `usage` totals (Chapter 28) against answers delivered.

COMMON MISTAKE

Backoff without jitter

Symptom. A service that recovers, is knocked over immediately, and recovers again, in a rhythm. **Cause.** A common-mode failure gives every client the same clock. **Fix.** `uniform(0, raw)` or `half + uniform(0, half)`. **Detection.** Panel C: a peak of **5,000/s** against 339, and 40 per cent served against 100 per cent.

COMMON MISTAKE

Retrying sooner than `retry-after` says

Symptom. 429s that never clear. **Cause.** The documentation is explicit that earlier retries will fail. **Fix.** Sleep exactly what the header says. **Detection.** Chapter 19's bake-off: **45,045 calls and 38,879 refusals** for the naive policy against **6,361 and zero** for the header-paced one.

COMMON MISTAKE

Relying on `max_retries` to bound fleet load

Symptom. Load multiplies during an outage even though every client is "only retrying twice". **Cause.** `max_retries` is per request; an incident is fleet-wide. **Fix.** A retry budget as a share of real traffic. **Detection.** Panel D: **3.00×** unbudgeted against **1.10×** budgeted at total failure.

COMMON MISTAKE

Retrying a refusal

Symptom. Three identical refusals and a user with no answer. **Cause.** A refusal is an HTTP 200 and the model's decision, not a transient condition. **Fix.** A different model, with fallback credit (Chapter 25). **Detection.** A refusal counter that is not an error counter.

COMMON MISTAKE

Treating a mid-stream error like a connection error

Symptom. A retry that regenerates content the user has already seen. **Cause.** An `error` event inside an HTTP 200 arrives after tokens have streamed and been produced. **Fix.** Chapter 17's recovery: resend the partial as a user message with an instruction to continue on 4.6 and later. **Detection.** Count `error` events separately from HTTP failures.

COMMON MISTAKE

Retrying a 409 with a sleep

Symptom. A conflict that never resolves. **Cause.** The documented advice is *"resolve the conflict, then retry"* — re-read state, do not merely wait. **Fix.** Refetch before the retry. **Detection.** A 409 whose retry count climbs monotonically.

COMMON MISTAKE

Generating the idempotency key inside the retry loop

Symptom. Duplicate rows despite an idempotency key. **Cause.** Each attempt gets a fresh key, so each attempt is idempotent only with itself. **Fix.** Generate once, before the first attempt. **Detection.** Two rows with different keys and identical content.

COMMON MISTAKE

A different retry policy at every call site

Symptom. Incidents that behave differently depending on which feature triggered them. **Cause.** Retry logic written where it was needed rather than once. **Fix.** One policy object in middleware. **Detection.** Grep for `sleep`, `backoff` and `max_retries`; count the answers.

COMMON MISTAKE

Preflighting nothing and retrying everything

Symptom. A steady background rate of 400s that the retry policy dutifully triples. **Cause.** Requests that Chapter 26's and Chapter 27's preflights would have refused offline, in microseconds, are being sent and retried instead. **Fix.** Preflight first. It is free. **Detection.** The 400 rate. It should be approximately zero.

29.22 Performance Considerations

A retry is the most expensive latency you will ever add, because it is paid *after* the request has already failed slowly. Chapter 26's worst case — three attempts at the SDK's 10-minute default — is **30 minutes**, and a user waiting 30 minutes for an error is a worse outcome than a fast failure.

Fail fast on non-retryable errors. A 400 that is retried twice costs three round trips to learn what the first one already said. This is the performance argument for the classifier, distinct from the capacity argument in Panel A.

Backoff should be bounded by something the user cares about. A per-request deadline — "this must answer within 90 seconds or not at all" — composes better than a retry count, because it bounds the thing you can actually promise.

Jitter costs nothing and buys the largest effect in the chapter. Panel C's 14.7× peak reduction is three lines of code and no additional latency in the common case.

Streaming changes the failure profile, not just the perception. A streamed request has no long idle period, so it is not exposed to the idle-connection drops the documentation warns about, and Chapter 17 measured the reassembly cost as negligible.

Batch removes the problem rather than tuning it. Chapter 30 covers it: no held connection, a 24-hour window, results retained for 29 days, and 50 per cent off input and output. For anything that does not need an answer in the next minute, it is strictly better than a well-tuned retry policy.

29.23 Security Considerations

A retry storm is a self-inflicted denial of service, and it is indistinguishable from an external one at the point where it matters. Panel D's 3.0× multiplier arrives exactly when capacity is scarce; a budget is a security control as much as a reliability one.

Duplicated effects are a correctness and safety problem, not only a cost one. If a completion triggers an action — Chapter 25's fourth layer, the only one that prevents harm — then a duplicated generation can mean a duplicated action. The idempotency key belongs on the **effect**, and it belongs there whether or not the model is involved.

Do not log request bodies on failure. The temptation during an incident is enormous and Chapter 26's leak scanner exists because logs outlive incidents. Log the `request-id`, the status, the attempt number and the model; nothing else.

Error messages are prose and may change. Chapter 26's rule: catch typed classes, never string-match. A retry policy keyed on message text will silently stop retrying after an unrelated wording change.

A 401 mid-run may mean a key expired, not that it was wrong. Chapter 26 recorded that expiration is set at creation and cannot be extended, and that short-lived keys expire with no warning email. A retry cannot fix it; alerting on `expires_at` can.

Retry budgets fail open by default and should not. If the counter is unavailable — a shared store is down — the safe behaviour is to deny retries, not to allow them, because the circumstances under which the counter is unavailable are exactly the circumstances in which retries are dangerous.

29.24 Real Production Examples

A summarisation service billed for five times what it delivered. The team had set `timeout=20.0` after reading the SDK page, on a workload whose median generation was around 45 seconds. Answers were delivered for fewer than half of requests, the API reported almost no errors, and the invoice tracked generations rather than answers. Raising the timeout to 300 seconds fixed both numbers in one deploy.

An outage that extended itself by nineteen minutes. A partial upstream failure resolved in about 30 seconds; the service took twenty minutes to recover. Every client backed off on the same schedule from the same instant, so the recovering capacity was consumed by a single burst each time and the remaining 90 per cent went round again. Adding equal jitter — three lines — removed the effect entirely in the next incident.

A bad deploy that exhausted the rate limit. A malformed field produced 400s on about a third of requests. The retry wrapper caught exceptions rather than classifying them, so each 400 became three, and the resulting RPM pressure produced 429s on the *healthy* two-thirds of traffic. The incident was reported as a rate-limit problem for its first hour.

Duplicate tickets from one support integration. A completion that created a ticket was retried after a client-side timeout. Both generations completed; both created a ticket. The key was being generated inside the retry loop, so each attempt was idempotent with respect to itself and nothing else.

29.25 Certification Notes

Concept	Domain	Weight	What is examinable
The status catalogue	D5 Context Management & Reliability	15%	which are retryable; 400/401/402/403/404/413 are not
Stop reasons vs errors	D5	15%	a refusal is an HTTP 200 and needs a different model
Client vs server timeouts	D5	15%	a client timeout does not cancel the generation
Idempotency	D5	15%	no documented key on `POST /v1/messages`

Concept	Domain	Weight	What is examinable
Backoff and jitter	D1 Agentic Architecture & Orchestration	27%	jitter is the part that works
`retry-after`	D5	15%	earlier retries will fail
SDK defaults	D2 Tool Design & MCP Integration	18%	`max_retries=2`, `timeout=600 s`, honours `retry-after`
Retry budgets	D1	27%	fleet-wide; `max_retries` cannot do it
Mid-stream errors	D2	18%	an `error` event inside an HTTP 200
Long requests	D5	15%	stream or batch; the 10-minute SDK guard

CERTIFICATION NOTE

Exam tips

Highest yield: **which statuses are retryable.** 408, 409, 429, 500, 504, 529 and a connection error — yes. 400, 401, 402, 403, 404, 413 — **no**. If an option says a 400 should be retried with backoff, it is wrong.

A refusal is not an error. It is an HTTP 200, and the remedy is a **different model**, not another attempt.

A client-side timeout does not cancel the request. Any question about paying for work you did not receive is asking about this.

`retry-after` **means exactly what it says** — *"earlier retries will fail."*

The SDK defaults are examinable numbers: **2 retries, 10-minute timeout**, retries on connection errors, 408, 409, 429 and ≥500.

Jitter, not the exponent, is what prevents a synchronised fleet. And `max_retries` **cannot bound fleet-wide load** — that needs a budget.

For long requests the documented answers are **streaming** and **Batch**, not a longer timeout.

29.26 Twenty Practice Questions

1. Which HTTP statuses does the documentation describe as retryable?
2. Which are not, and why not?
3. What does a connection error tell you about whether the request was processed?
4. Is a refusal an error? What is the correct response?
5. What happens on the server when your client-side timeout expires?
6. Does `POST /v1/messages` accept an idempotency key?
7. What are the SDK defaults for retries and timeout?
8. Which errors do the SDKs retry by default?
9. What does the documentation say about retrying earlier than `retry-after`?
10. What is the difference between a 429 and a 529?
11. Why does 409 need different handling from 429?

12. What is jitter, and what problem does it solve?
13. Why does a fleet without jitter send more requests and serve fewer clients?
14. What is a retry budget and why can `max_retries` not provide one?
15. What are the four costs of a retry?
16. What should you do about a `max_tokens` stop reason?
17. How does an error inside an HTTP 200 differ from a connection error?
18. What is the documented advice for requests expected to take more than ten minutes?
19. Where should an idempotency key be generated, and why?
20. What should a classifier do with a status it does not recognise?

29.27 Answers and Explanations

1. **408, 409, 429, 500, 504, 529**, plus connection errors with no status.
2. **400, 401, 402, 403, 404, 413.** The cause does not change with time — a malformed body is still malformed, an oversized request is still oversized.
3. **Nothing.** No response was received, so the request may or may not have been processed. It is retryable and it is the case where a retry is most likely to duplicate work.
4. **No — it is an HTTP 200 with `stop_reason: "refusal"`.** The documented remedy is a retry on a **different model**, ideally with fallback credit.
5. **Nothing.** The generation runs to completion. Closing the connection is a decision your process makes; the server is not informed.
6. **The documentation describes none.** It is documented neither as supported nor as unsupported, so treat it as UNKNOWN and assume a retry is a second request.
7. `max_retries=2` **and** `timeout=600 seconds` (10 minutes).
8. **Connection errors, 408, 409, 429 and ≥500**, with a short exponential backoff, honouring `retry-after`.
9. That **earlier retries will fail.** The header replaces your backoff calculation.
10. **429 is your rate limit; 529 is Anthropic overloaded across all users.** Both are retryable; only the 429 carries a `retry-after` you must respect, and only the 429 can be prevented by pacing your own traffic.
11. Because the documented fix is *"resolve the conflict, then retry"* — re-read the state. Sleeping alone will conflict again.
12. **Randomisation of the backoff delay.** It prevents a fleet that failed together from retrying together.
13. Because each synchronised burst exceeds capacity, so most of it fails and retries again. Panel C: **42,000** requests for 40 per cent served, against 34,998 for 100 per cent.
14. A cap on **total** retries as a share of real traffic. `max_retries` bounds one request; an incident is fleet-wide, so a hundred well-behaved clients still produce two hundred retries.
15. **Latency, rate limit, money, and a duplicated effect.** Only the fourth has no dial.
16. Not retry it. **Raise** `max_tokens` — sized for thinking plus text (Chapter 27) — or continue the response.

17. A mid-stream error arrives **after** tokens have been produced and streamed. There is partial output and it has been generated; a retry is a fresh generation, and Chapter 17's continuation rules apply.
18. **Stream it, or use the Batch API.** Not a longer timeout.
19. **Before the first attempt, outside the retry loop**, so that every attempt shares it. Generating it inside makes each attempt idempotent only with itself.
20. **Raise.** The documented `type` values may grow, and defaulting an unknown status to non-retryable fails silently.

29.28 Ten Practical Exercises

1. Find every retry in your codebase and count how many distinct policies you have.
2. Add a classifier and measure the drop in retry traffic over a week.
3. Plot your own generation-latency distribution and mark your configured timeout on it.
4. Compute generations-per-answer at your current timeout.
5. Add jitter to one backoff and re-run your load test.
6. Add a per-process retry budget and drive it to total failure in a test.
7. Alert on p100 latency exceeding your configured timeout.
8. Count refusals separately from errors and confirm the counter is not always zero.
9. Take one call that writes something and move its idempotency key outside the retry loop.
10. Pick one long-running workload and move it to streaming or Batch.

29.29 Five Challenge Labs

1. **The deadline propagator.** Replace retry counts with an end-to-end deadline that is passed down through every layer, so a call that cannot finish in time is never started.
2. **The shared budget.** Implement a fleet-wide retry budget over a shared counter, and decide — and test — what happens when the counter is unavailable.
3. **The duplicate detector.** Reconcile `usage` totals from Chapter 28 against answers delivered, and produce a daily generations-per-answer figure for your own service.
4. **The jitter chooser.** Simulate your own outage profile and backoff schedule and determine whether full or equal jitter is better for you. Publish the crossover.
5. **The chaos harness.** Inject each of the thirteen documented failures into a staging environment and assert that your handler does the right thing for every one, including raising on an unlisted status.

29.30 Ten Interview Questions

1. Which failures do you retry, and which do you not?
2. A request times out on the client. What happened on the server?
3. How would you find out whether your retries are costing you money?
4. Why is jitter more important than the backoff exponent?

5. `max_retries=2` on every client. What is your fleet's load multiplier at a 50 per cent failure rate?
6. A refusal comes back. Walk me through the handling.
7. How do you make a non-idempotent API safe to retry?
8. What is the difference between a 429 and a 529 in how you respond?
9. Your service takes twenty minutes to recover from a thirty-second outage. Diagnose it.
10. When would you replace retries with Batch?

29.31 Summary

Chapter 28 handled the branch where the call succeeds. This one handles the branch where it does not, and its argument is that the word "retry" hides the question that matters: **what did the first attempt already do?**

The taxonomy comes first, because a third of the problem is solved by reading it. Thirteen documented outcomes: **six retryable** — 408, 409, 429, 500, 504 and 529 — **six not** — 400, 401, 402, 403, 404 and 413 — and a connection error with no status at all, which is retryable and is the one case where you cannot know whether the request was processed. A policy that retries anything that threw sends **20,000** requests for **10,000** errors and wastes **6,600 of them, 33.0 per cent**, on failures that were arithmetically incapable of changing. Those wasted requests arrive during an incident, at the highest possible rate, into the rate-limit bucket Chapter 19 showed is shared with real traffic. And the catalogue is open, so a classifier that returns "not retryable" for an unfamiliar status is guessing silently; the one here raises.

Three things that look like failures are not, and each needs a different response. A **stop reason** is a successful HTTP 200 — `max_tokens` wants a bigger cap, not the same request again. A **refusal** is a successful HTTP 200 and the one condition a retry on the same model cannot improve; Chapter 25's fallback, with credit, is the remedy. And an **error event inside a stream** arrives after tokens have been produced, so it is neither an error nor a completion, and Chapter 17's continuation rules apply.

Then the measurement that gives the chapter its title. **A client-side timeout does not cancel the generation.** Closing a socket is a decision your process makes and the server is never told, so every abandoned attempt runs to completion. On a workload with a 45-second median, the SDK documentation's own `timeout=20.0` example produces **5.439 completed generations for every answer delivered**, with **54.36 per cent** of logical requests returning nothing at all; at the SDK's 600-second default the same workload runs **1.002** generations per answer. The curve between them is a cliff at roughly the p90, which is why the timeout has to come from a measured distribution rather than an example. And the duplication cannot be deduplicated, because **no idempotency key is documented on** `POST /v1/messages` — documented neither way, therefore UNKNOWN, therefore assume the expensive case and make the **effect** idempotent instead.

Finally, two properties of retrying that are invisible from inside a single client. **Jitter is not a refinement of backoff; it is the part that works** — five thousand clients through a thirty-second outage serve **40.0 per cent** of themselves while sending **42,000** requests without it, and **100 per cent** while sending **34,998** with equal jitter, a **14.7×** reduction in peak arrival rate. And **a retry budget is fleet-wide where** `max_retries` **is per request**: at a 30 per cent failure rate the load multiplier is **1.6027×** unbudgeted and **1.1000×** budgeted, and under a total outage **3.00× against 1.10×** — invisible when things are fine,

decisive when they are not.

KEY TAKEAWAYS

1. **Six statuses are retryable and six are not.** A naive policy wastes **33.0 per cent** of its retries on failures that cannot change.
2. **Raise on an unrecognised status.** The documented `type` values may grow, and a silent default fails silently.
3. **A stop reason is not an error, and a refusal needs a different model**, not another attempt at the same one.
4. **A client-side timeout does not cancel the generation.** Every abandoned attempt completes on the server.
5. **The documentation's `timeout=20.0` example produced 5.439 generations per answer** and a 54.36 per cent failure rate on a 45-second-median workload.
6. **No idempotency key is documented.** Assume a retry is a second request and make the effect idempotent, with a key generated before the first attempt.
7. **Jitter is the part that works** — 14.7× lower peak, 100 per cent served instead of 40 per cent, while sending fewer requests.
8. **`retry-after` replaces your backoff.** Earlier retries will fail.
9. **`max_retries` cannot bound fleet load.** A budget holds a total outage at 1.10× instead of 3.00×.
10. **A retry has four meters** — latency, rate limit, money and a duplicated effect — and only the last has no dial.

29.32 Glossary

Term	Definition
Backoff	The delay before a retry, usually `base × factor^(n−1)` and capped.
Common-mode failure	A failure that hits every client at once, which is what makes jitter necessary.
Connection error	A failure with no HTTP status; the request may or may not have been processed.
Equal jitter	Half the raw delay plus a uniform draw over the other half.
Full jitter	A uniform draw between zero and the raw delay.
Idempotency key	A caller-supplied identifier that makes a repeated request a no-op. **Not documented on `POST /v1/messages`.**
Jitter	Randomisation of the backoff delay, to desynchronise a fleet.
`max_retries`	The SDK's per-request retry count; **2** by default.
Non-retryable	A failure whose cause does not change with time — 400, 401, 402, 403, 404, 413.
Retry budget	A fleet-wide cap on retries as a share of real traffic.
`retry-after`	A response header giving the earliest time a retry can succeed; earlier retries will fail.

Term	Definition
Retryable	A failure whose cause changes on its own — 408, 409, 429, 500, 504, 529, connection errors.
Thundering herd	A synchronised fleet arriving together and exceeding capacity in an instant.
Timeout	Client-side (10 minutes by default in the SDKs) or server-side (504); **the client's does not cancel the work**.

29.33 Further Reading

Primary sources, verified on 1 August 2026 at `platform.claude.com/docs`:

- `api/errors` — the status catalogue, the error body shape, typed SDK exceptions, request IDs, and the long-requests guidance.
- `api/rate-limits` — 429s, `retry-after`, acceleration limits and the token bucket (Chapter 19).
- `cli-sdks-libraries/sdks/python` — `max_retries`, `timeout`, the 10-minute `ValueError` guard and TCP keep-alive (Chapter 26).
- `build-with-claude/handling-stop-reasons` — why a stop reason is not an error (Chapter 28).
- `build-with-claude/refusals-and-fallback` and `build-with-claude/fallback-credit` — the correct response to a refusal (Chapter 25).
- `build-with-claude/streaming` — `error` events inside an HTTP 200 and the recovery rules (Chapter 17).
- `build-with-claude/batch-processing` — the alternative to holding a connection open (Chapter 30).

Where to go next in this book. Chapter 30 closes Part 5 with Batch, prompt caching and cost controls, and is the structural answer to most of this chapter's problems: no held connection, no idle-timeout exposure, and 50 per cent off. **Chapters 31–35** build the tool-use loop, where a retry has to be reasoned about per step rather than per request. **Chapter 51** covers replanning after a failed step, and **Chapter 56** treats failure modes — circuit breakers, bulkheads and load shedding — as a production discipline in their own right, with this chapter's budget as one instrument among several.